Universidad Autónoma de Madrid
Escuela Politécnica Superior

Fundamentos de Criptografía y Seguridad Informática

Shamir's Secret Sharing

Doble grado Ingeniería Informática - Matemáticas

Autor:

Rodrigo Alcántara Fernández rodrigo.alcantara@estudiante.uam.es

Fecha: 1 de noviembre de 2026



Índice

1. Introducción 2
2. El polinomio interpolador de Lagrange 2
3. Cómo compartir un secreto 7
4. Corrección y seguridad del sistema 8
5. Ejemplo 10
6. En la práctica 12
7. Implementación 13
Bibliografía 15

1. Introducción

El esquema de compartición de secretos es un protocolo definido en 1979 por Adi Shamir [1] (uno de los inventores de RSA). Busca dividir un secreto entre un grupo de personas de tal manera que se pueda recuperar si y solo si un subgrupo de tamaño prefijado de los integrantes del grupo se ponen de acuerdo. Por ejemplo, se puede compartir entre un grupo de 10 personas y solo se puede recuperar el secreto si 7 de ellas así lo quieren.

2. El polinomio interpolador de Lagrange

Para desarrollar este apartado me he basado en los apuntes de la asignatura de Cálculo Numérico II [2].

Sea $\mathbb{K}$ un cuerpo (e.g. $\mathbb{R}$ o $\mathbb{Z}_p$). Sean $N \in \mathbb{N}$, $x_0, x_1, ..., x_N \in \mathbb{K}$ puntos distintos dos a dos y $y_0, y_1, ..., y_N \in \mathbb{K}$. El problema de interpolación de Lagrange consiste en buscar un polinomio $q(x) \in \mathbb{K}[x]$ de grado menor o igual a N tal que $q(x_i) = y_i \quad \forall i \in \{0, 1, ..., N\}$.

Definición 2.1 (Nodos de interpolación)

A los puntos x_i los llamamos «nodos de interpolación».

♣

Definición 2.2 (Polinomios de Lagrange)

Dada una colección $\{x_i\}_{i=0}^N$ de nodos de interpolación, definimos el i-ésimo polinomio de Lagrange como:

$$l_i(x) = \prod_{\substack{k=0,\\ i\neq k}}^{N} \frac{x - x_k}{x_i - x_k} = \frac{(x - x_0)...(x - x_{i-1})(x - x_{i+1})...(x - x_N)}{(x_i - x_0)...(x_i - x_{i-1})(x_i - x_{i+1})...(x_i - x_N)}. \quad (1)$$

♣

Observación 2.3

Los denominadores en (1) nunca se anulan porque los nodos son distintos dos a dos.

♢

Proposición 2.4 (Valor de los polinomios de Lagrange en los nodos interpoladores)

$$l_{i(x_j)} = \delta_{i,j} = \begin{cases} 1 \text{ si } i = j \\ 0 \text{ si } i \neq j \end{cases}. \tag{2}$$

♠

Demostración.

- $i = j$:

$$l_{i(x_i)} = \prod_{\substack{k=0, \\ i \neq k}}^{N} \frac{x_i - x_k}{x_i - x_k} = \prod_{\substack{k=0, \\ i \neq k}}^{N} 1 = 1. \quad \checkmark$$

- $i \neq j$: el numerador contiene el factor $(x_j - x_j) = 0$, por lo que el producto es 0:

$$\begin{aligned} l_{i(x_j)} &= \prod_{\substack{k=0, \\ i \neq k}}^{N} \frac{x_j - x_k}{x_i - x_k} \\ &= \frac{(x_j - x_0)...\cancel{(x_j - x_j)}...(x_j - x_{i-1})(x_j - x_{i+1})...(x_j - x_N)}{(x_i - x_0)...(x_i - x_{i-1})(x_i - x_{i+1})...(x_i - x_N)} \\ &= 0. \quad \checkmark \end{aligned}$$

□

Teorema 2.5 (Polinomio Interpolador de Lagrange)

Dados $N + 1$ nodos de interpolación y $N + 1$ valores $y_0, y_1, ..., y_N$, el polinomio:

$$q(x) = \sum_{i=0}^{N} l_i(x) y_i, \tag{3}$$

es un polinomio de grado menor o igual a N tal que $q(x_i) = y_i \quad \forall i \in \{0, 1, ..., N\}$.

♡

Demostración.

- Es un polinomio de grado menor o igual a N:
  Está construido como la suma de polinomios Lagrange, que a su vez son el producto de N factores lineales, luego el grado de $q(x)$ es, a lo sumo, N. ✓

- $q(x_i) = y_i$:

$$\begin{aligned} q(x_i) &= \sum_{j=0}^{N} l_j(x_i) y_j \\ &= \cancel{l_0(x_i)} y_0 + ... + \cancel{l_{i-1}(x_i)} y_{i-1} + l_i(x_i) y_i + \cancel{l_{i+1}(x_i)} y_{i+1} + ... + \cancel{l_N(x_i)} y_N \\ &= 0 \cdot y_0 + ... + 0 \cdot y_{i-1} + 1 \cdot y_i + 0 \cdot y_{i+1} + ... + 0 \cdot y_N \\ &= y_i. \quad \checkmark \end{aligned}$$

□

Lema 2.6

Un polinomio $q(x)$ no nulo de grado menor o igual que N sobre un cuerpo $\mathbb{K}$ tiene como mucho N raíces.

♡

Demostración. Por inducción en N.

- Si $N = 0$, $q(x)$ es constante (no nulo) y por lo tanto no tiene raíces. ✓

Suponemos ahora que es cierto para $N-1$, y sea $q(x) \neq 0$ de grado $\leq N$.

Si q no tiene raíces, hemos terminado. Si $a \in \mathbb{K}$ es una raíz, dividiendo por $(x-a)$ obtenemos $q(x) = (x-a)s(x)$ con $s(x)$ de grado menor o igual a $N-1$ y $s \neq 0$.

Si $b \neq a$ es otra raíz, entonces $0 = q(b) = (b-a)s(b)$, y como $\mathbb{K}$ es un cuerpo y $b - a \neq 0$, entonces $s(b) = 0$. Por hipótesis de inducción, s tiene como mucho $N-1$ raíces, luego q tiene como mucho N. ✓ □

Teorema 2.7 (Unicidad del polinomio interpolador)

Dados $N+1$ puntos $x_0, x_1, ..., x_N \in \mathbb{K}$ distintos dos a dos y $N+1$ valores $y_0, y_1, ..., y_N \in \mathbb{K}$, existe un único polinomio $q(x) \in \mathbb{K}[x]$ de grado menor o igual a N tal que:

$$q(x_i) = y_i \quad \forall i \in \{0, 1, ..., N\}. \tag{4}$$

♡

Demostración.

- La existencia del polinomio la tenemos por el Teorema 2.5. ✓
- Unicidad: supongamos que existen dos polinomios $p(x)$ y $q(x)$, de grado menor o igual a N, tales que cumplen:

$$p(x_i) = q(x_i) = y_i \quad \forall i \in \{0, 1, ..., N\}.$$

Definimos $r(x) \coloneqq p(x) - q(x)$, que por ser la resta de dos polinomios de grado menor o igual a N debe ser también de grado menor o igual a N. Sin embargo,

$$r(x_i) = p(x_i) - q(x_i) = y_i - y_i = 0 \quad \forall i \in \{0, 1, ..., N\},$$

por lo que $r(x)$ tiene al menos $N + 1$ raíces, pero es de grado menor o igual que N, luego por el Lema 2.6 solo puede ser el polinomio idénticamente igual a 0.

$$r(x) = 0 = p(x) - q(x) \Rightarrow p(x) = q(x) \Rightarrow \text{el polinomio interpolador es único.} \quad ✓$$

□

Ejemplo 2.8

Calcular el polinomio cúbico que pasa por los puntos $(1, 1), (2, 1), (3, 2), (4, 6)$. ♢

En primer lugar definimos los polinomios de Lagrange:

$$l_0(x) = \frac{(x-2)(x-3)(x-4)}{(1-2)(1-3)(1-4)} = \frac{x^3 - 9x^2 + 26x - 24}{-6} = -\frac{1}{6}x^3 + \frac{3}{2}x^2 - \frac{13}{3}x + 4,$$

$$l_1(x) = \frac{(x-1)(x-3)(x-4)}{(2-1)(2-3)(2-4)} = \frac{x^3 - 8x^2 + 19x - 12}{2} = \frac{1}{2}x^3 - 4x^2 + \frac{19}{2}x - 6,$$

$$l_2(x) = \frac{(x-1)(x-2)(x-4)}{(3-1)(3-2)(3-4)} = \frac{x^3 - 7x^2 + 14x - 8}{-2} = -\frac{1}{2}x^3 + \frac{7}{2}x^2 - 7x + 4,$$

$$l_3(x) = \frac{(x-1)(x-2)(x-3)}{(4-1)(4-2)(4-3)} = \frac{x^3 - 6x^2 + 11x - 6}{6} = \frac{1}{6}x^3 - x^2 + \frac{11}{6}x - 1.$$

Ahora calculamos $q(x)$:

$$\begin{aligned} q(x) &= \sum_{i=0}^{3} l_i(x) y_i = l_0(x) + l_1(x) + 2l_2(x) + 6l_3(x) \\ &= \left(-\frac{1}{6}x^3 + \frac{3}{2}x^2 - \frac{13}{3}x + 4\right) + \left(\frac{1}{2}x^3 - 4x^2 + \frac{19}{2}x - 6\right) \\ &\quad +2\left(-\frac{1}{2}x^3 + \frac{7}{2}x^2 - 7x + 4\right) + 6\left(\frac{1}{6}x^3 - x^2 + \frac{11}{6}x - 1\right) \\ &= \frac{1}{3}x^3 - \frac{3}{2}x^2 + \frac{13}{6}x. \end{aligned}$$

Se puede comprobar fácilmente:

$$\begin{aligned} q(1) &= \frac{1}{3} - \frac{3}{2} + \frac{13}{6} = 1, \quad \checkmark \\ q(2) &= \frac{1}{3} \cdot 8 - \frac{3}{2} \cdot 4 + \frac{13}{6} \cdot 2 = 1, \quad \checkmark \\ q(3) &= \frac{1}{3} \cdot 27 - \frac{3}{2} \cdot 9 + \frac{13}{6} \cdot 3 = 2, \quad \checkmark \\ q(4) &= \frac{1}{3} \cdot 64 - \frac{3}{2} \cdot 16 + \frac{13}{6} \cdot 4 = 6. \quad \checkmark \end{aligned}$$

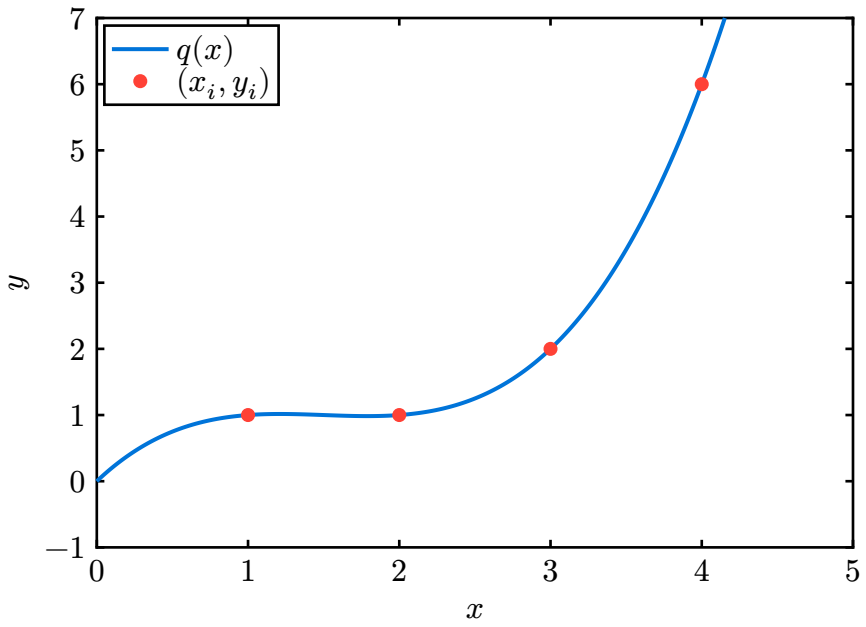


Figura 1: El polinomio interpolador y sus nodos de interpolación

3. Cómo compartir un secreto

Observación 3.1

Sin pérdida de generalidad podemos asumir que el secreto que queremos compartir es (o se puede convertir en) un número entero. Además, vamos a trabajar en un cuerpo finito $\mathbb{F}_{p^m}$ con p un número primo y p^m mayor que el secreto que se quiere compartir. En la Proposición 4.2 veremos que esto es necesario para obtener secreto perfecto.

♢

Definición 3.2 (Umbral)

El umbral del esquema de compartición se define como una tupla (k, n) donde n es el número de trozos en los que se ha repartido el secreto y k es el número mínimo de trozos necesarios para recuperarlo. n está limitado por el orden del cuerpo sobre el que estemos trabajando: $n < p^m$.

♣

El primer paso es decidir el umbral de nuestro reparto, decidiendo cuántos trozos se van a repartir y cuántos van a ser capaces de reconstruir el secreto. Una vez que ya tenemos n y k lo que vamos a buscar es convertir el secreto en un polinomio $q(x)$ de grado menor o igual a $k-1$.

Para ello, elegimos aleatoriamente $a_1, a_2, ..., a_{k-1} \in \mathbb{F}$ de manera uniforme e independiente, y ponemos a_0 igual al secreto (que por la Observación 3.1 es un número). De esta manera, conocer el polinomio nos da el secreto, ya que basta evaluarlo en $x = 0$ para obtener su término independiente.

Teniendo el polinomio (de grado menor o igual a $k-1$) vamos a obtener n puntos, por ejemplo: $q(1), q(2), q(3), ..., q(n)$. Ahora podemos repartir estos puntos (que son los «trozos» del secreto).

Por lo visto en la Sección 2, con k trozos podemos obtener un polinomio de grado menor o igual que $k-1$ que pase por esos k puntos. Además, por el Teorema 2.7 sabemos que el polinomio es único. Como nuestro polinomio $q(x)$ era de grado menor o igual que $k-1$ y pasaba por esos puntos, necesariamente el polinomio interpolador es exactamente el polinomio $q(x)$. Hemos recuperado el secreto a partir de k trozos.

4. Corrección y seguridad del sistema

Ya hemos visto que con k trozos es posible recuperar el secreto. Pero, ¿qué ocurre si más de k trozos intentan reconstruir el secreto? Y si conocemos $k' < k$ trozos, ¿tenemos algún ataque con posibilidad de recuperar el secreto?

Proposición 4.1

El sistema tiene corrección. ♠

Demostración. Sean:

- $S \in \mathbb{F}$ el secreto que se quiere compartir,
- (k, n) el umbral del sistema,
- $q(x) \in \mathbb{F}[x]$ de grado menor o igual que $k-1$ escogido como clave y tal que $q(0) = S$,
- $\{(x_i, q(x_i))\}_{i \in \{1,\dots,n\}}$ los trozos repartidos.

Caso 1: k trozos

Por el Teorema 2.7 sabemos que, dados k trozos distintos cualesquiera, existe un único polinomio $r(x)$ de grado menor o igual que $k-1$ que pasa por esos puntos. Como el polinomio original $q(x)$ tiene grado menor o igual a $k-1$, y pasa por los k trozos, sabemos que estos dos polinomios q y r deben ser iguales, sin importar qué trozos han sido escogidos. Por lo tanto, es posible recuperar el secreto. ✓

Caso 2: k' trozos, con $k' > k$

En el caso de tener más trozos, al calcular el polinomio interpolador nos va a salir un polinomio $r(x)$ de grado menor o igual que $k'-1$ que pase por los k' nodos. Como $q(x)$ tiene grado menor o igual a $k-1$ en particular tiene grado menor que $k'-1$, ya que $k < k'$. Además, $q(x)$ pasa por los k' nodos, por lo que es un polinomio interpolador de estos nodos. Por el Teorema 2.7 el polinomio interpolador es único, luego q y r deben ser iguales. Por lo tanto, es posible recuperar el secreto. ✓ □

Proposición 4.2

$\forall \mathbb{F}$ cuerpo finito, si $|\mathbb{F}| > n$ el sistema tiene seguridad perfecta. ♠

Demostración. Sea $\mathbb{F} = \mathbb{F}_{p^m}$. Queremos ver que, aunque tengamos acceso a $k' < k$ trozos, no tenemos ninguna información extra acerca del secreto, es decir: $\forall s \in \mathbb{F}$

$$\Pr[S = s] = \Pr[S = s \mid \text{trozos}],$$

donde S es una v.a. sobre los posibles secretos y s un secreto.

En primer lugar podemos ver que es suficiente con probarlo para $k-1$ trozos, ya que cualquier conjunto de $k' < k$ está contenido en un conjunto de $k-1$ trozos, por lo que si el secreto es independiente de estos también lo es de cualquier subconjunto suyo.

Sean $(x_1, y_1), ..., (x_{k-1}, y_{k-1})$ los trozos conocidos. Fijamos $s \in \mathbb{F}$. Cada posible elección de los coeficientes $(a_1, ..., a_{k-1}) \in \mathbb{F}^{k-1}$ nos da un polinomio $q(x)$ de grado menor o igual a $k-1$ con $q(0) = s$, por lo que tenemos $(p^m)^{k-1}$ posibilidades, todas equiprobables con probabilidad $(p^m)^{-(k-1)}$.

Los trozos son los que hemos visto si y solo si $q(x_i) = y_i \quad \forall i \in \{1, 2, ..., k-1\}$, es decir, si q pasa por los k puntos $(0, s)$, (x_1, y_1),..., (x_{k-1}, y_{k-1}). Por el Teorema 2.7, existe exactamente un polinomio de grado menor o igual que $k-1$ que cumple esto. Por lo tanto, solo una de las elecciones de $(a_1, ..., a_{k-1})$ es compatible, y

$$\Pr[\text{trozos} \mid S = s] = (p^m)^{-(k-1)} \quad \forall s \in \mathbb{F}.$$

Por la fórmula de la probabilidad total:

$$\Pr[\text{trozos}] = \sum_{s \in \mathbb{F}} \Pr[\text{trozos} \mid S = s] \Pr[S = s] = (p^m)^{-(k-1)} \sum_{s \in \mathbb{F}} \Pr[S = s] = (p^m)^{-(k-1)}.$$

Luego:

$$\Pr[S = s \mid \text{trozos}] = \frac{\Pr[\text{trozos} \mid S = s] \Pr[S = s]}{\Pr[\text{trozos}]} = \frac{(p^m)^{-(k-1)} \Pr[S = s]}{(p^m)^{-(k-1)}} = \Pr[S = s]. \quad \checkmark$$

□

5. Ejemplo

Observación 5.1

En la realidad habría que hacerlo en un cuerpo finito ($\mathbb{Z}_p$ o $\mathbb{F}_{p^n}$), pero por simplicidad lo voy a hacer en $\mathbb{R}$.

♢

Ejemplo 5.2

Queremos repartir el secreto 54321 entre 10 personas, de tal manera que 7 de ellas sean capaces de recuperarlo.

♢

1. Definimos el umbral: $(k, n) = (7, 10)$.

2. Creamos un polinomio de grado $k-1$ con término independiente igual al secreto:

$$q(x) = 54321 - 120x + 2544x^2 - 329x^3 + 85x^4 - 18x^5 + x^6$$

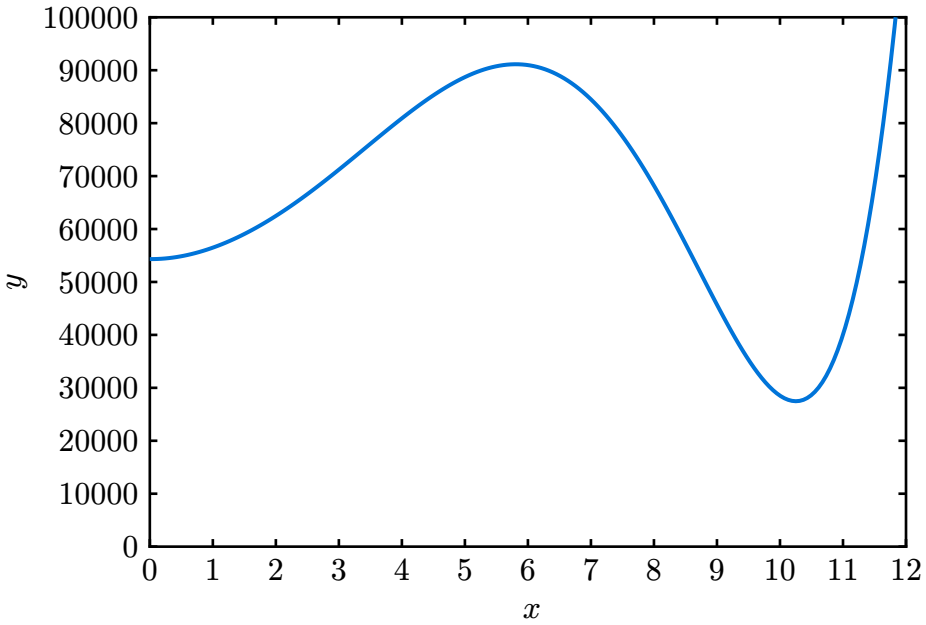


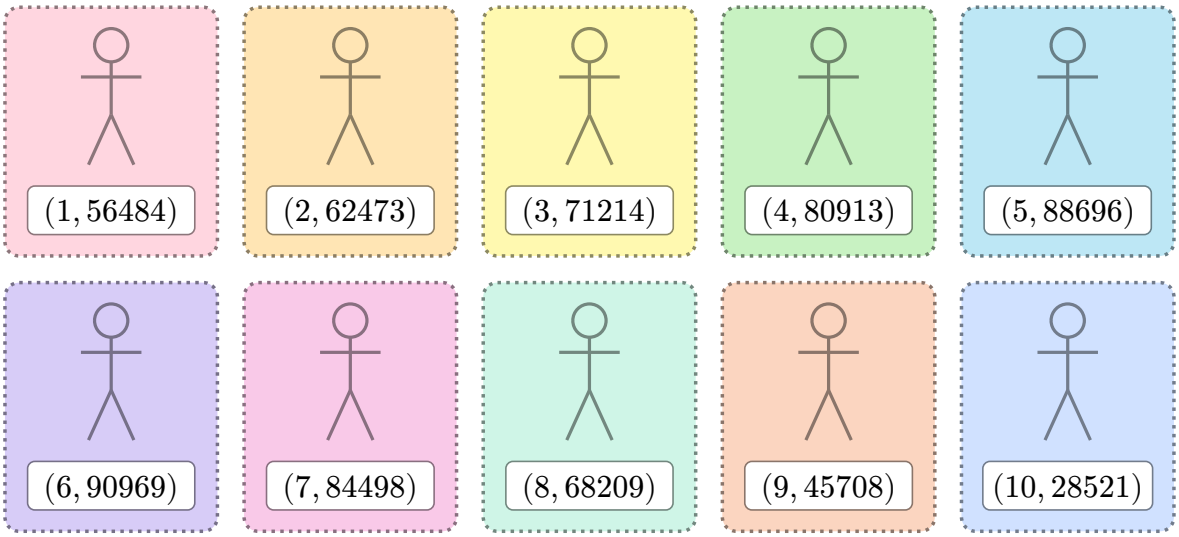
Figura 2: El polinomio $q(x)$, que almacena la clave en $q(0)$

3. Ahora escogemos 10 puntos del polinomio:

x	0	1	2	3	4	5	6	7	8	9	10
$q(x)$	54321	56484	62473	71214	80913	88696	90969	84498	68209	45708	28521

Tabla 1: Evaluación del polinomio en 10 puntos distintos del 0

4. Ya tenemos todos los ingredientes, ahora solo hay que repartir los trozos. Cada trozo consiste en un par $(x_i, q(x_i))$, según la Tabla 1.



5. Supongamos ahora que 7 de ellos (e.g. 1, 3, 5, 6, 7, 8 y 10) quieren recuperar el secreto. Para ello, construyen el polinomio interpolador de sus nodos mediante el método de Lagrange que hemos visto:

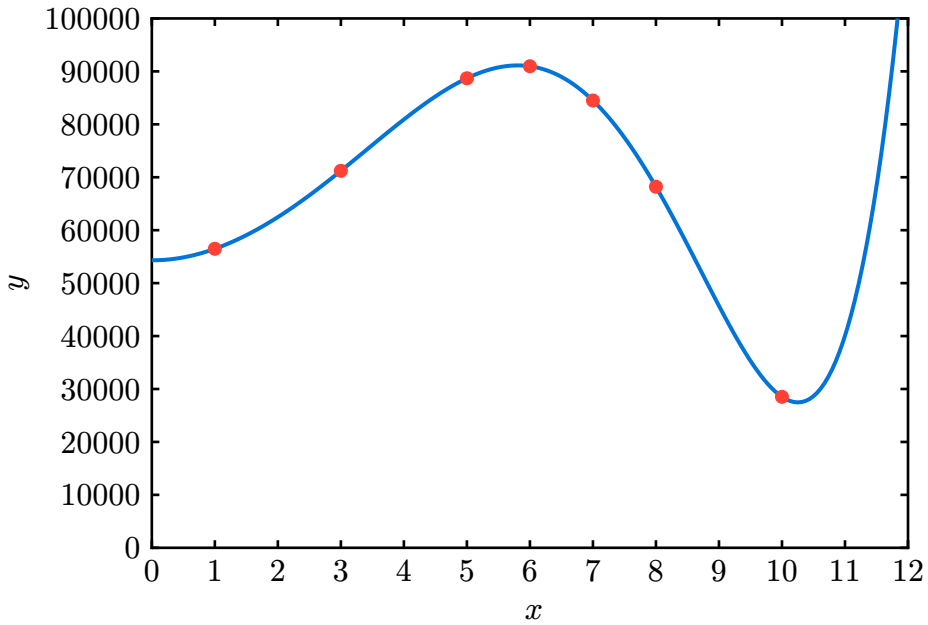


Figura 3: El polinomio interpolador de los nodos 1, 3, 5, 6, 7, 8 y 10, que coincide con $q(x)$

6. Una vez que se conoce el polinomio interpolador, basta evaluarlo en $x = 0$ para obtener el secreto: $q(0) = 54321$.

6. En la práctica

A continuación expongo una serie de consideraciones sobre la aplicación práctica de este esquema de compartición:

1. Hemos demostrado que este sistema tiene seguridad perfecta cuando trabajamos sobre un cuerpo finito, lo que nos viene bien a la hora de implementar esto en un ordenador. Este cuerpo tiene que ser de orden p^m mayor que el secreto y mayor que n (para garantizar que a cada persona se le puede dar un trozo distinto). Es decir: $p^m > \max(n, s)$.

2. Como normalmente lo que se quiere compartir es un documento o cualquier fichero binario en general, una opción bastante común es cifrar ese fichero con un sistema estándar y después compartir la clave de descifrado mediante este esquema de compartición de secretos. De tal manera que todo el mundo recibe el archivo cifrado y un trozo de la clave.

3. Se puede hacer un reparto desigual, dependiendo de las necesidades. Por ejemplo, podemos imaginar una empresa que quiere compartir un secreto (e.g. una clave criptográfica para firmar documentos) entre sus directivos, de tal manera que hagan falta 5 miembros de la junta para acceder a la clave. Sin embargo, también se quiere permitir que el presidente y un único directivo puedan tener la capacidad de acceder al secreto. Para lograr esto, basta con darle 4 trozos al presidente, y uno a cada directivo.

4. Este mecanismo tiene dos ventajas principales:
   - No hace falta disponer de todos los trozos para acceder al secreto, evitando así el riesgo de perder algún trozo (hasta $n - k$) o que pueda haber alguna persona que traicione al equipo y bloquee el descifrado.
   - Hace falta que haya varias personas (k) que quieran acceder de forma deshonesta al secreto, ya que hacen falta k trozos para descifrarlo.

   En función de las necesidades, se pueden ajustar los parámetros del umbral para adecuarlo a la situación.

5. Algunas limitaciones de este esquema son:
   - Se necesita un agente confiable para llevar a cabo el proceso: existe la posibilidad de que el encargado de realizar los trozos se guarde algún trozo extra, o directamente el secreto, ya que conoce toda la información. También se debe confiar en que al descifrar el secreto, solo se usa de manera correcta. Por ejemplo, si el secreto es una clave RSA para firmar, la idea es que tras recuperarla se use y luego

se borre, garantizando así que la próxima vez que se use sea porque al menos k trozos están de acuerdo.

- Un participante puede entregar un trozo falso, que según el método propuesto no se puede comprobar, y provocar que el secreto recuperado no sea el correcto. Esto lo soluciona Feldman en [3] con su esquema verificable.

7. Implementación

Nota

Para escribir el código he usado Inteligencia Artificial.

Link al repositorio: https://github.com/rodalc24/shamir-secret-sharing

Se puede compilar usando el comando `make`. El binario generado es `bin/sss` y para ejecutar los tests se puede usar `make test`.

♢

Para ilustrar el funcionamiento de este esquema, he implementado una versión de *Shamir's Secret Sharing* que permite compartir archivos binarios. Para ello hace uso de la biblioteca *openssl* y cifra el archivo con AES-256-GCM y después reparte la clave.

```
shamir@host:~$ ./sss -h
Usage:
  sss encrypt <file> -k <threshold> -n <shares> [-o <prefix>]
      Writes <prefix>.enc and <prefix>.share1 ... <prefix>.share<n>
      (prefix defaults to <file>). Any <threshold> shares can decrypt.
  sss decrypt <file.enc> <share>... [-o <output>]
      Each <share> is a share file, or the share text itself (SSS1-...).
      Output defaults to <file.enc> without .enc; use -o - for stdout.
```

Cifrado

El proceso de cifrado es el siguiente:

1. Genera una clave de 32 *bytes* aleatoria de manera segura (i.e. usando `RAND_bytes`).
2. Cifra el archivo con esa clave mediante AES-256-GCM y genera un `.enc`.
3. Aplica el esquema de *Shamir's Secret Sharing* a la clave, de acuerdo con los parámetros de `n` y `k` proporcionados. La clave se divide *byte* a *byte*, considerando cada uno de los 32 *bytes* como un secreto. Para cada *byte* se construye un polinomio aleatorio en $\mathbb{F}_{2^8}$, con el término constante igual al secreto. Después se evalúa el polinomio en n puntos, asignándole uno a cada trozo.
4. Escribe cada uno de los trozos en un fichero.
5. Borra de manera segura la clave, el contenido del archivo y los trozos de la memoria (con `OPENSSL_cleanse`) para evitar posibles filtraciones.

```
shamir@host:~$ ls -1
file.pdf
sss
shamir@host:~$ ./sss encrypt file.pdf -k 3 -n 5
Encrypted "file.pdf" -> "file.pdf.enc" Wrote 5 shares (file.pdf.share1 … file.pdf.share5);
any 3 of them can decrypt.
shamir@host:~$ ls -1
file.pdf
file.pdf.enc
file.pdf.share1
file.pdf.share2
file.pdf.share3
file.pdf.share4
file.pdf.share5
sss
shamir@host:~$ cat file.pdf.share*
SSS1-3-01f5455964317263336c4dfc08b6c3d4ef124cc6c81c572df5d566c9257e4ecce1-796e039b
SSS1-3-020856ad6d1e4ce4dd1f22a2ad97b103a355cbff26f1654a4dc5bf5bc967127e1f-30ebc40b
SSS1-3-03b39154388338cc1b888cd1812019277ef7144b5c26132517094db65f8b41e8e1-508abf8d
SSS1-3-046e2e077ca92cdf6f4154d44497b6d7cbbffaa61ab53f2733ddbf1468bb915eae-7ec6d5b0
SSS1-3-05d5e9fe293458f7a9d6faa768201ef3161d25126062494869114df9fe57c2c850-76571f90
```

Cada trozo está formado por las siguientes partes:

$$\underbrace{\text{SSS1}}_{\text{Versión}} - \underbrace{3}_{k} - \underbrace{01}_{x} \underbrace{\text{f5}}_{q_1(x)} \underbrace{45}_{q_2(x)} \underbrace{59}_{q_3(x)} \ldots \underbrace{\text{e1}}_{q_{32}(x)} - \underbrace{\text{796e039b}}_{\text{checksum}}$$

donde la x es el punto asignado a cada trozo (coincide con su índice) empezando en 1. Cada *byte* posterior es el previamente llamado y_i, uno por cada *byte* de la clave que se ha repartido. El *checksum* se usa para detectar errores y modificaciones del trozo (e.g. de la k), pero no protege frente a una modificación maliciosa.

Descifrado

El proceso de descifrado es el siguiente:

1. Lee los trozos, comprobando el *checksum* y que la k coincide. También mira que la longitud es correcta (33 *bytes*) y que hay un número mayor o igual que k de trozos con distinta x.
2. Reconstruye la clave a partir de los trozos.
3. Descifra el archivo con la clave usando AES-256-GCM.

```
shamir@host:~$ ls -1
file.pdf
file.pdf.enc
file.pdf.share1
file.pdf.share2
file.pdf.share3
file.pdf.share4
file.pdf.share5
sss
shamir@host:~$ ./sss decrypt file.pdf.enc file.pdf.share* -o file2.pdf
Decrypted "file.pdf.enc" -> "file2.pdf" (used 3 of 5 given shares)
shamir@host:~$ ls -1
file.pdf
file.pdf.enc
file.pdf.share1
file.pdf.share2
file.pdf.share3
file.pdf.share4
file.pdf.share5
file2.pdf
sss
shamir@host:~$ diff -s file.pdf file2.pdf
Files file.pdf and file2.pdf are identical
```

Bibliografía

[1] A. Shamir, «How to Share a Secret», *Communications of the ACM*, vol. 22, n.º 11, pp. 612-613, nov. 1979, doi: 10.1145/359168.359176.

[2] J. Novo, «Interpolación Polinómica de Lagrange».

[3] P. Feldman, «A practical scheme for non-interactive verifiable secret sharing», en *28th Annual Symposium on Foundations of Computer Science (sfcs 1987)*, IEEE, 1987, pp. 427-438.